My Project

Generated by Doxygen 1.9.8

1 Class Index 1

1.1 Class List . . . 1

2 File Index 3

2.1 File List . . . 3

3 Class Documentation 5

3.1 ressources_s Struct Reference . . . 5

3.1.1 Detailed Description . . . 5

3.1.2 Member Data Documentation . . . 5

3.1.2.1 background . . . 5

3.1.2.2 background2 . . . 6

3.1.2.3 font . . . 6

3.1.2.4 level_1_screen . . . 6

3.1.2.5 level_2_screen . . . 6

3.1.2.6 ligne_arrivee . . . 6

3.1.2.7 meteorite . . . 6

3.1.2.8 missile . . . 6

3.1.2.9 ovni . . . 6

3.1.2.10 spacecraft . . . 7

3.1.2.11 splash_text . . . 7

3.1.2.12 start_screen . . . 7

3.2 sprite_s Struct Reference . . . 7

3.2.1 Detailed Description . . . 7

3.2.2 Member Data Documentation . . . 7

3.2.2.1 h . . . 7

3.2.2.2 w . . . 8

3.2.2.3 x . . . 8

3.2.2.4 y . . . 8

3.3 world_s Struct Reference . . . 8

3.3.1 Detailed Description . . . 9

3.3.2 Member Data Documentation . . . 9

3.3.2.1 game_status . . . 9

3.3.2.2 gameover . . . 9

3.3.2.3 invisible_limit . . . 9

3.3.2.4 ligne_arrivee . . . 9

3.3.2.5 missileCooldown . . . 9

3.3.2.6 missiles . . . 9

3.3.2.7 ovnis_level_1 . . . 10

3.3.2.8 ovnis_level_2 . . . 10

3.3.2.9 spacecraft . . . 10

3.3.2.10 splash_text . . . 10

3.3.2.11 time . . . 10

3.3.2.12 vy . . . 10
3.3.2.13 walls_level_1 . . . 10
3.3.2.14 walls_level_2 . . . 10
3.3.2.15 win . . . 10

4 File Documentation **11**
4.1 affichage.c File Reference . . . 11
4.2 affichage.h File Reference . . . 11
4.2.1 Detailed Description . . . 12
4.2.2 Function Documentation . . . 12
4.2.2.1 apply_background_level_1() . . . 12
4.2.2.2 apply_background_level_2() . . . 13
4.2.2.3 apply_level_1_screen() . . . 13
4.2.2.4 apply_level_2_screen() . . . 13
4.2.2.5 apply_ovnis() . . . 13
4.2.2.6 apply_sprites1() . . . 14
4.2.2.7 apply_startscreen() . . . 14
4.2.2.8 apply_texts() . . . 14
4.2.2.9 apply_walls() . . . 15
4.2.2.10 clean_ressources() . . . 15
4.2.2.11 init_textures() . . . 15
4.2.2.12 refresh_graphics() . . . 15
4.3 affichage.h . . . 16
4.4 constantes.h File Reference . . . 16
4.4.1 Detailed Description . . . 17
4.5 constantes.h . . . 18
4.6 logique.c File Reference . . . 18
4.6.1 Detailed Description . . . 19
4.6.2 Function Documentation . . . 19
4.6.2.1 blink_splash_text() . . . 19
4.6.2.2 handle_sprites_collision() . . . 20
4.6.2.3 init_data() . . . 20
4.6.2.4 init_ovnis() . . . 20
4.6.2.5 init_sprite() . . . 20
4.6.2.6 init_walls() . . . 21
4.6.2.7 is_finished_game() . . . 21
4.6.2.8 is_finished_level() . . . 21
4.6.2.9 shoot_missile() . . . 22
4.6.2.10 sprites_collide() . . . 22
4.6.2.11 update_data() . . . 22
4.6.2.12 update_finishline() . . . 22
4.6.2.13 update_missileCooldown() . . . 23

4.6.2.14 update_missiles() . . . 23
4.6.2.15 update_ovnis() . . . 23
4.6.2.16 update_time() . . . 23
4.6.2.17 update_walls() . . . 24
4.7 logique.h File Reference . . . 24
4.7.1 Detailed Description . . . 25
4.7.2 Function Documentation . . . 25
4.7.2.1 blink_splash_text() . . . 25
4.7.2.2 handle_sprites_collision() . . . 26
4.7.2.3 init_data() . . . 26
4.7.2.4 init_ovnis() . . . 26
4.7.2.5 init_sprite() . . . 27
4.7.2.6 init_walls() . . . 27
4.7.2.7 is_finished_game() . . . 27
4.7.2.8 is_finished_level() . . . 27
4.7.2.9 shoot_missile() . . . 28
4.7.2.10 sprites_collide() . . . 28
4.7.2.11 update_data() . . . 28
4.7.2.12 update_finishline() . . . 28
4.7.2.13 update_missileCooldown() . . . 30
4.7.2.14 update_missiles() . . . 30
4.7.2.15 update_ovnis() . . . 30
4.7.2.16 update_time() . . . 30
4.7.2.17 update_walls() . . . 31
4.8 logique.h . . . 31
4.9 main.c File Reference . . . 32
4.9.1 Detailed Description . . . 32
4.9.2 Function Documentation . . . 33
4.9.2.1 clean() . . . 33
4.9.2.2 handle_events() . . . 33
4.9.2.3 init() . . . 33
4.9.2.4 skip_to_next_stage() . . . 34
4.10 sdl2-light.c File Reference . . . 34
4.10.1 Detailed Description . . . 34
4.10.2 Function Documentation . . . 35
4.10.2.1 apply_texture() . . . 35
4.10.2.2 clean_sdl() . . . 35
4.10.2.3 clean_texture() . . . 35
4.10.2.4 clear_renderer() . . . 36
4.10.2.5 init_sdl() . . . 36
4.10.2.6 load_image() . . . 36
4.10.2.7 pause() . . . 37

4.10.2.8 update_screen() . . . 37
4.11 sdl2-light.h File Reference . . . 37
4.11.1 Detailed Description . . . 38
4.11.2 Function Documentation . . . 38
4.11.2.1 apply_texture() . . . 38
4.11.2.2 clean_sdl() . . . 39
4.11.2.3 clean_texture() . . . 39
4.11.2.4 clear_renderer() . . . 39
4.11.2.5 init_sdl() . . . 39
4.11.2.6 load_image() . . . 40
4.11.2.7 pause() . . . 40
4.11.2.8 update_screen() . . . 40
4.12 sdl2-light.h . . . 41
4.13 sdl2-ttf-light.h . . . 41

Index **43**

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ressources_s
: Représentation pour stocker les ressources nécessaires à l'affichage graphique 5

sprite_s
: Type qui correspond au sprite 7

world_s
: Représentation du monde du jeu 8

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

affichage.c
: Contient l'ensemble des fonctions relatives à l'affichage des éléments du jeu 11

affichage.h
: Header de affichage.c 11

constantes.h
: Contient l'ensemble des constantes du jeu 16

logique.c
: Contient l'ensemble des fonctions relatives à la logique du jeu 18

logique.h
: Header de logique.c 24

main.c
: Programme principal du jeu 32

sdl2-light.c
: Sur-couche de SDL2 pour simplifier son utilisation pour le projet 34

sdl2-light.h
: En-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet 37

sdl2-ttf-light.h 41

Chapter 3

Class Documentation

3.1 ressources_s Struct Reference

Représentation pour stocker les ressources nécessaires à l'affichage graphique.

`#include <affichage.h>`

Public Attributes

- SDL_Texture * start_screen
- SDL_Texture * splash_text
- SDL_Texture * background
- SDL_Texture * background2
- SDL_Texture * level_1_screen
- SDL_Texture * level_2_screen
- SDL_Texture * spacecraft
- SDL_Texture * ligne_arrivee
- SDL_Texture * meteorite
- SDL_Texture * missile
- SDL_Texture * ovni
- TTF_Font * font

3.1.1 Detailed Description

Représentation pour stocker les ressources nécessaires à l'affichage graphique.

3.1.2 Member Data Documentation

3.1.2.1 background

`SDL_Texture* ressources_s::background`

Texture liée à l'image du fond de l'écran du niveau 1

3.1.2.2 background2

```
SDL_Texture* ressources_s::background2
```

Texture liée à l'image du fond de l'écran du niveau 2

3.1.2.3 font

```
TTF_Font* ressources_s::font
```

Texture liée à la police du texte

3.1.2.4 level_1_screen

```
SDL_Texture* ressources_s::level_1_screen
```

Texture liée à l'image de l'écran titre du niveau 1

3.1.2.5 level_2_screen

```
SDL_Texture* ressources_s::level_2_screen
```

Texture liée à l'image de l'écran titre du niveau 2

3.1.2.6 ligne_arrivee

```
SDL_Texture* ressources_s::ligne_arrivee
```

Texture liée à l'image de la ligne d'arrivée

3.1.2.7 meteorite

```
SDL_Texture* ressources_s::meteorite
```

Texture liée à l'image d'une météorite

3.1.2.8 missile

```
SDL_Texture* ressources_s::missile
```

Texture liée à l'image d'un missile

3.1.2.9 ovni

```
SDL_Texture* ressources_s::ovni
```

Texture liée à l'image d'un ovni

3.1.2.10 spacecraft

```
SDL_Texture* ressources_s::spacecraft
```

Texture liée à l'image du vaisseau

3.1.2.11 splash_text

```
SDL_Texture* ressources_s::splash_text
```

Texture liée au texte PRESS SPACE TO START

3.1.2.12 start_screen

```
SDL_Texture* ressources_s::start_screen
```

Texture liée à l'image de l'écran titre

The documentation for this struct was generated from the following file:

- affichage.h

3.2 sprite_s Struct Reference

Type qui correspond au sprite.

```
#include <logique.h>
```

Public Attributes

- float x
- float y
- float w
- float h

3.2.1 Detailed Description

Type qui correspond au sprite.

3.2.2 Member Data Documentation

3.2.2.1 h

```
float sprite_s::h
```

Champ indiquant la hauteur h du sprite

3.2.2.2 w

```
float sprite_s::w
```

Champ indiquant la largeur w du sprite

3.2.2.3 x

```
float sprite_s::x
```

Champ indiquant la position x du sprite

3.2.2.4 y

```
float sprite_s::y
```

Champ indiquant la position y du sprite

The documentation for this struct was generated from the following file:

- logique.h

3.3 world_s Struct Reference

Représentation du monde du jeu.

```
#include <logique.h>
```

Collaboration diagram for world_s:

Public Attributes

- sprite_t splash_text
- sprite_t spacecraft
- sprite_t ligne_arrivee
- sprite_t walls_level_1 [6]
- sprite_t walls_level_2 [50]
- sprite_t ovnis_level_1 [3]
- sprite_t ovnis_level_2 [30]
- sprite_t missiles [10]
- sprite_t invisible_limit
- int game_status
- float vy
- int gameover
- int win
- float time
- unsigned int missileCooldown

3.3.1 Detailed Description

Représentation du monde du jeu.

3.3.2 Member Data Documentation

3.3.2.1 game_status

```
int world_s::game_status
```

Champ indiquant si le statut actuel du jeu

3.3.2.2 gameover

```
int world_s::gameover
```

Champ indiquant si on a perdu

3.3.2.3 invisible_limit

```
sprite_t world_s::invisible_limit
```

Champ représentant le sprite de la limite invisible (pour stopper les missiles)

3.3.2.4 ligne_arrivee

```
sprite_t world_s::ligne_arrivee
```

Champ représentant le sprite de la ligne d'arrivée

3.3.2.5 missileCooldown

```
unsigned int world_s::missileCooldown
```

Champ représentant le cooldown restant du tir de missile (en ms)

3.3.2.6 missiles

```
sprite_t world_s::missiles[10]
```

Champ représentant le tableau des sprites des missiles tirés par le vaisseau

3.3.2.7 ovnis_level_1

```
sprite_t world_s::ovnis_level_1[3]
```

Champ représentant le tableau du sprite des ovnis du niveau 1

3.3.2.8 ovnis_level_2

```
sprite_t world_s::ovnis_level_2[30]
```

Champ représentant le tableau du sprite des ovnis du niveau 2

3.3.2.9 spacecraft

```
sprite_t world_s::spacecraft
```

Champ représentant le sprite du vaisseau

3.3.2.10 splash_text

```
sprite_t world_s::splash_text
```

Champ représentant le sprite du texte PRESS SPACE TO START

3.3.2.11 time

```
float world_s::time
```

Champ représentant le temps écoulé depuis le début du jeu

3.3.2.12 vy

```
float world_s::vy
```

Champ représentant la vitesse de déplacement de la ligne d'arrivée

3.3.2.13 walls_level_1

```
sprite_t world_s::walls_level_1[6]
```

Champ représentant le tableau du sprite des murs de météorites du niveau 1

3.3.2.14 walls_level_2

```
sprite_t world_s::walls_level_2[50]
```

Champ représentant le tableau du sprite des murs de météorites du niveau 2

3.3.2.15 win

```
int world_s::win
```

Champ indiquant si on a gagné

The documentation for this struct was generated from the following file:

- logique.h

Chapter 4

File Documentation

4.1 affichage.c File Reference

Contient l'ensemble des fonctions relatives à l'affichage des éléments du jeu.

```
#include "affichage.h"
#include "logique.h"
#include "constantes.h"
```

Include dependency graph for affichage.c:

4.2 affichage.h File Reference

Header de affichage.c.

```
#include "sdl2-light.h"
#include "sdl2-ttf-light.h"
#include "logique.h"
```

Include dependency graph for affichage.h: This graph shows which files directly or indirectly include this file:

Classes

- struct ressources_s

  Représentation pour stocker les ressources nécessaires à l'affichage graphique.

Typedefs

- typedef struct ressources_s **ressources_t**

  Représentation pour stocker les ressources nécessaires à l'affichage graphique.

Functions

- void init_textures (SDL_Renderer ∗renderer, ressources_t ∗ressources)

  La fonction initialise les textures.
- void apply_walls (ressources_t ∗ressources, SDL_Renderer ∗renderer, world_t ∗world)

  fonction qui applique la texture des murs de météorites (en fonction du niveau)
- void apply_ovnis (ressources_t ∗ressources, SDL_Renderer ∗renderer, world_t ∗world)

  fonction qui applique la texture des ovnis (en fonction du niveau)
- void apply_sprites1 (SDL_Renderer ∗renderer, ressources_t ∗ressources, world_t ∗world)

  La fonction applique la texture des sprites sur le renderer lié à l'écran de jeu.
- void apply_startscreen (SDL_Renderer ∗renderer, ressources_t ∗ressources)

  La fonction applique la texture de l'écran de démarrage syr le renderer lié à l'écran de jeu.
- void apply_level_1_screen (SDL_Renderer ∗renderer, ressources_t ∗ressources)

  La fonction applique la texture de l'écran titre du niveau 1 sur le renderer lié à l'écran du jeu.
- void apply_level_2_screen (SDL_Renderer ∗renderer, ressources_t ∗ressources)

  La fonction applique la texture de l'écran titre du niveau 2 sur le renderer lié à l'écran du jeu.
- void apply_background_level_1 (SDL_Renderer ∗renderer, ressources_t ∗ressources)

  La fonction applique la texture du fond sur le renderer lié à l'écran du jeu (niveau 1)
- void apply_background_level_2 (SDL_Renderer ∗renderer, ressources_t ∗ressources)

  La fonction applique la texture du fond sur le renderer lié à l'écran du jeu (niveau 2)
- void apply_texts (SDL_Renderer ∗renderer, world_t ∗world, ressources_t ∗ressources)

  La fonction affiche le timer et les messages de fin sur le renderer lié à l'écran de jeu.
- void refresh_graphics (SDL_Renderer ∗renderer, world_t ∗world, ressources_t ∗ressources)

  La fonction rafraichit l'écran en fonction de l'état des données du monde.
- void clean_ressources (ressources_t ∗ressources)

  La fonction nettoie les ressources.

4.2.1 Detailed Description

Header de affichage.c.

Author

Valentini Andréa & Viale Romain

4.2.2 Function Documentation

4.2.2.1 apply_background_level_1()

```
void apply_background_level_1 (
            SDL_Renderer * renderer,
            ressources_t * ressources )
```

La fonction applique la texture du fond sur le renderer lié à l'écran du jeu (niveau 1)

Parameters

renderer	le renderer
ressources	les ressources du jeu

4.2.2.2 apply_background_level_2()

```
void apply_background_level_2 (
            SDL_Renderer * renderer,
            ressources_t * ressources )
```

La fonction applique la texture du fond sur le renderer lié à l'écran du jeu (niveau 2)

Parameters

renderer	le renderer
ressources	les ressources du jeu

4.2.2.3 apply_level_1_screen()

```
void apply_level_1_screen (
            SDL_Renderer * renderer,
            ressources_t * ressources )
```

La fonction applique la texture de l'écran titre du niveau 1 sur le renderer lié à l'écran du jeu.

Parameters

renderer	le renderer
ressources	les ressources du jeu

4.2.2.4 apply_level_2_screen()

```
void apply_level_2_screen (
            SDL_Renderer * renderer,
            ressources_t * ressources )
```

La fonction applique la texture de l'écran titre du niveau 2 sur le renderer lié à l'écran du jeu.

Parameters

renderer	le renderer
ressources	les ressources du jeu

4.2.2.5 apply_ovnis()

```
void apply_ovnis (
            ressources_t * ressources,
            SDL_Renderer * renderer,
            world_t * world )
```

fonction qui applique la texture des ovnis (en fonction du niveau)

Parameters

ressources	les ressources du jeu
renderer	le renderer
world	le monde du jeu

4.2.2.6 apply_sprites1()

```
void apply_sprites1 (
            SDL_Renderer * renderer,
            ressources_t * ressources,
            world_t * world )
```

La fonction applique la texture des sprites sur le renderer lié à l'écran de jeu.

Parameters

renderer	le renderer
ressources	les ressources du jeu
world	le monde du jeu

4.2.2.7 apply_startscreen()

```
void apply_startscreen (
            SDL_Renderer * renderer,
            ressources_t * ressources )
```

La fonction applique la texture de l'écran de démarrage syr le renderer lié à l'écran de jeu.

Parameters

renderer	le renderer
ressources	les ressources du jeu

4.2.2.8 apply_texts()

```
void apply_texts (
            SDL_Renderer * renderer,
            world_t * world,
            ressources_t * ressources )
```

La fonction affiche le timer et les messages de fin sur le renderer lié à l'écran de jeu.

Parameters

renderer	le renderer
world	le monde du jeu
ressources	les ressources du jeu

4.2.2.9 apply_walls()

```
void apply_walls (
            ressources_t * ressources,
            SDL_Renderer * renderer,
            world_t * world )
```

fonction qui applique la texture des murs de météorites (en fonction du niveau)

Parameters

ressources	les ressources du jeu
renderer	le renderer
world	le monde du jeu

4.2.2.10 clean_ressources()

```
void clean_ressources (
            ressources_t * ressources )
```

La fonction nettoie les ressources.

Parameters

ressources	les ressources

4.2.2.11 init_textures()

```
void init_textures (
            SDL_Renderer * renderer,
            ressources_t * ressources )
```

La fonction initialise les textures.

Parameters

renderer	le renderer
ressources	les ressources du jeu

4.2.2.12 refresh_graphics()

```
void refresh_graphics (
            SDL_Renderer * renderer,
            world_t * world,
            ressources_t * ressources )
```

La fonction rafraichit l'écran en fonction de l'état des données du monde.

Parameters

renderer	le renderer
world	le monde du jeu
ressources	les ressources du jeu

4.3 affichage.h

Go to the documentation of this file.

```
00001
00007 #ifndef AFFICHAGE_H
00008 #define AFFICHAGE_H
00009
00010 #include "sdl2-light.h"
00011 #include "sdl2-ttf-light.h"
00012
00016 typedef struct ressources_s{
00017     SDL_Texture* start_screen;
00018     SDL_Texture* splash_text;
00019     SDL_Texture* background;
00020     SDL_Texture* background2;
00021     SDL_Texture* level_1_screen;
00022     SDL_Texture* level_2_screen;
00023     SDL_Texture* spacecraft;
00024     SDL_Texture* ligne_arrivee;
00025     SDL_Texture* meteorite;
00026     SDL_Texture* missile;
00027     SDL_Texture* ovni;
00028     TTF_Font* font;
00029 } ressources_t;
00030
00031 #include "logique.h" // plus bas pour éviter les dépendances circulaires
00032
00038 void init_textures(SDL_Renderer *renderer, ressources_t *ressources);
00039
00046 void apply_walls(ressources_t* ressources, SDL_Renderer* renderer, world_t* world);
00047
00054 void apply_ovnis(ressources_t* ressources, SDL_Renderer* renderer, world_t* world);
00055
00062 void apply_sprites1(SDL_Renderer *renderer , ressources_t *ressources , world_t * world);
00063
00069 void apply_startscreen(SDL_Renderer* renderer, ressources_t* ressources);
00070
00076 void apply_level_1_screen(SDL_Renderer* renderer, ressources_t* ressources);
00077
00083 void apply_level_2_screen(SDL_Renderer* renderer, ressources_t* ressources);
00084
00090 void apply_background_level_1(SDL_Renderer *renderer, ressources_t *ressources);
00091
00097 void apply_background_level_2(SDL_Renderer* renderer, ressources_t* ressources);
00098
00105 void apply_texts(SDL_Renderer *renderer, world_t *world, ressources_t *ressources);
00106
00113 void refresh_graphics(SDL_Renderer *renderer, world_t *world, ressources_t *ressources);
00114
00119 void clean_ressources(ressources_t *ressources);
00120
00121 #endif
```

4.4 constantes.h File Reference

Contient l'ensemble des constantes du jeu.

This graph shows which files directly or indirectly include this file:

Macros

- #define **SCREEN_WIDTH** 300

  Largeur de l'écran de jeu.
- #define **SCREEN_HEIGHT** 480

  Hauteur de l'écran de jeu.
- #define **SPLASH_TEXT_TIME** 1000

  Durée visible/caché à l'écran du texte "PRESS SPACE TO START!".
- #define **TRANSITION_TIME** 2000

  Durée d'affichage de l'écran de titre des niveaux.
- #define **SHIP_SIZE** 32

  Taille d'un vaisseau.
- #define **METEORITE_SIZE** 32

  Taille d'une météorite.
- #define **OVNI_WIDTH** 27

  Largeur d'un ovni.
- #define **OVNI_HEIGHT** 19

  Hauteur d'un ovni.
- #define **MISSILE_WIDTH** 9

  Largeur d'un missile.
- #define **MISSILE_HEIGHT** 17

  Hauteur d'un missile.
- #define **FINISH_LINE_HEIGHT_1** -960

  Hauteur de la ligne d'arrivée du niveau 1.
- #define **FINISH_LINE_HEIGHT_2** -7000

  Hauteur de la ligne d'arrivée du niveau 2.
- #define **INVISIBLE_LIMIT_HEIGHT** -100

  Hauteur de la limite verticale invisible du jeu.
- #define **MOVING_STEP** 10

  Pas de déplacement horizontal du vaisseau.
- #define **MISSILE_SPEED** 4

  Vitesse d'un missile.
- #define **MISSILE_COOLDOWN** 500

  Cooldown d'un missile (en ms)
- #define **INITIAL_SPEED** 1

  Vitesse initiale de déplacement vertical des éléments du jeu.

4.4.1 Detailed Description

Contient l'ensemble des constantes du jeu.

Author

Valentini Andréa & Viale Romain

4.5 constantes.h

Go to the documentation of this file.

```
00001
00007 #ifndef CONSTANTES_H
00008 #define CONSTANTES_H
00009
00013 #define SCREEN_WIDTH 300
00014
00018 #define SCREEN_HEIGHT 480
00019
00023 #define SPLASH_TEXT_TIME 1000
00024
00028 #define TRANSITION_TIME 2000
00029
00033 #define SHIP_SIZE 32
00034
00038 #define METEORITE_SIZE 32
00039
00043 #define OVNI_WIDTH 27
00044
00048 #define OVNI_HEIGHT 19
00049
00053 #define MISSILE_WIDTH 9
00054
00058 #define MISSILE_HEIGHT 17
00059
00063 #define FINISH_LINE_HEIGHT_1 -960
00064
00068 #define FINISH_LINE_HEIGHT_2 -7000
00069
00073 #define INVISIBLE_LIMIT_HEIGHT -100
00074
00078 #define MOVING_STEP 10
00079
00083 #define MISSILE_SPEED 4
00084
00088 #define MISSILE_COOLDOWN 500
00089
00093 #define INITIAL_SPEED 1
00094
00095 #endif
```

4.6 logique.c File Reference

Contient l'ensemble des fonctions relatives à la logique du jeu.

```
#include "logique.h"
#include "constantes.h"
#include <stdlib.h>
```

Include dependency graph for logique.c:

Functions

- void init_sprite (sprite_t *sprite, int x, int y, int w, int h)

  La fonction initialise la position du sprite.
- void init_walls (world_t *world)

  La fonction initialise la position des murs (en fonction du niveau)
- void init_ovnis (world_t *world)

  La fonction initialise la position des ovnis (en fonction du niveau)
- void init_data (world_t *world)

  La fonction initialise les données du monde (en fonction du niveau)
- void blink_splash_text (world_t *world, unsigned int timeSinceLaunch)

  Fait clignoter le texte "PRESS SPACE TO START!" en fonction de la constante SPLASH_TEXT_TIME.
- int sprites_collide (sprite_t *sp1, sprite_t *sp2)

La fonction renvoie l'état de collision de 2 sprites (0 ou 1)

- void handle_sprites_collision (world_t *world, sprite_t *sp1, sprite_t *sp2, int make_disappear, int make_win)

  La fonction gère les collisions entre deux sprites.
- int is_finished_level (world_t *world)

  Renvoie si le niveau est terminé ou non.
- int is_finished_game (world_t *world)

  Renvoie si le jeu est terminé ou non.
- void update_time (world_t *world, unsigned int actualTime, unsigned int *lastTimeCheck)

  Met à jour le temps écoulé
- void update_finishline (world_t *world)

  Met à jour la position de la ligne d'arrivée + gère les collisions.
- void update_walls (world_t *world)

  Met à jour la position des murs de météorites + gère les collisions (en fonction du niveau)
- void update_ovnis (world_t *world)

  Met à jour la position des ovnis + gère les collisions (en fonction du niveau)
- void shoot_missile (world_t *world)

  Tire un missile à partir du vaisseau.
- void update_missiles (world_t *world)

  Met à jour la position du missile + gère les collisions (en fonction du niveau)
- void update_missileCooldown (world_t *world, unsigned int actualTime, unsigned int *lastMissileCooldown↩ Check)

  Met à jour le temps restant au cooldown du tir de missile du vaisseau.
- void update_data (world_t *world, ressources_t *ressources)

  La fonction met à jour les données en tenant compte de la physique du monde.

4.6.1 Detailed Description

Contient l'ensemble des fonctions relatives à la logique du jeu.

Author

Valentini Andréa & Viale Romain

4.6.2 Function Documentation

4.6.2.1 blink_splash_text()

```
void blink_splash_text (
            world_t * world,
            unsigned int timeSinceLaunch )
```

Fait clignoter le texte "PRESS SPACE TO START!" en fonction de la constante SPLASH_TEXT_TIME.

Parameters

world	les données du monde
timeSinceLaunch	la durée depuis laquelle le jeu est lancé (en ms)

4.6.2.2 handle_sprites_collision()

```
void handle_sprites_collision (
            world_t * world,
            sprite_t * sp1,
            sprite_t * sp2,
            int make_disappear,
            int make_win )
```

La fonction gère les collisions entre deux sprites.

Parameters

world	les données du monde
sp1	le premier sprite
sp2	le deuxième sprite
make_disappear	1 si sp1 doit disparaître en cas de collision, 2 si sp1 et sp2 doivent disparaitre, 0 sinon
make_win	1 si la collision déclenche la victoire, 0 si défaite, autre valeur entière si ça ne doit rien faire
ressources	les ressources du jeu

4.6.2.3 init_data()

```
void init_data (
            world_t * world )
```

La fonction initialise les données du monde (en fonction du niveau)

Parameters

world	les données du monde

4.6.2.4 init_ovnis()

```
void init_ovnis (
            world_t * world )
```

La fonction initialise la position des ovnis (en fonction du niveau)

Parameters

world	les données du monde

4.6.2.5 init_sprite()

```
void init_sprite (
            sprite_t * sprite,
            int x,
```

```
            int y,
            int w,
            int h )
```

La fonction initialise la position du sprite.

Parameters

sprite	le sprite du jeu
x	la position x du sprite
y	la position y du sprite
w	la largeur du sprite
h	la hauteur du sprite

4.6.2.6 init_walls()

```
void init_walls (
            world_t * world )
```

La fonction initialise la position des murs (en fonction du niveau)

Parameters

world	les données du monde

4.6.2.7 is_finished_game()

```
int is_finished_game (
            world_t * world )
```

Renvoie si le jeu est terminé ou non.

Parameters

world	les données du monde

4.6.2.8 is_finished_level()

```
int is_finished_level (
            world_t * world )
```

Renvoie si le niveau est terminé ou non.

Parameters

world	les données du monde

4.6.2.9 shoot_missile()

```
void shoot_missile (
            world_t * world )
```

Tire un missile à partir du vaisseau.

Parameters

world	les données du monde

4.6.2.10 sprites_collide()

```
int sprites_collide (
            sprite_t * sp1,
            sprite_t * sp2 )
```

La fonction renvoie l'état de collision de 2 sprites (0 ou 1)

Parameters

sp1	le premier sprite
sp2	le deuxième sprite

4.6.2.11 update_data()

```
void update_data (
            world_t * world,
            ressources_t * ressources )
```

La fonction met à jour les données en tenant compte de la physique du monde.

Parameters

world	les données du monde
ressources	les ressources du jeu

4.6.2.12 update_finishline()

```
void update_finishline (
            world_t * world )
```

Met à jour la position de la ligne d'arrivée + gère les collisions.

Parameters

world	les données du monde

4.6.2.13 update_missileCooldown()

```
void update_missileCooldown (
            world_t * world,
            unsigned int actualTime,
            unsigned int * lastMissileCooldownCheck )
```

Met à jour le temps restant au cooldown du tir de missile du vaisseau.

Parameters

world	les données du monde
actualTime	le temps écoulé depuis le lancement du jeu (en ms)
lastMissileCooldownCheck	le temps écoulé au dernier check (en ms)

4.6.2.14 update_missiles()

```
void update_missiles (
            world_t * world )
```

Met à jour la position du missile + gère les collisions (en fonction du niveau)

Parameters

world	les données du monde

4.6.2.15 update_ovnis()

```
void update_ovnis (
            world_t * world )
```

Met à jour la position des ovnis + gère les collisions (en fonction du niveau)

Parameters

world	les données du monde

4.6.2.16 update_time()

```
void update_time (
            world_t * world,
            unsigned int actualTime,
            unsigned int * lastTimeCheck )
```

Met à jour le temps écoulé

Parameters

world	les données du monde
actualTime	le temps écoulé depuis le lancement du jeu (en ms)
lastTimeCheck	le temps écoulé au dernier check (en ms)

4.6.2.17 update_walls()

```
void update_walls (
            world_t * world )
```

Met à jour la position des murs de météorites + gère les collisions (en fonction du niveau)

Parameters

world	les données du monde

4.7 logique.h File Reference

Header de logique.c.

```
#include "sdl2-light.h"
#include "affichage.h"
```

Include dependency graph for logique.h: This graph shows which files directly or indirectly include this file:

Classes

- struct sprite_s

  Type qui correspond au sprite.
- struct world_s

  Représentation du monde du jeu.

Typedefs

- typedef struct sprite_s **sprite_t**

  Type qui correspond au sprite.
- typedef struct world_s **world_t**

  Représentation du monde du jeu.

Functions

- void init_sprite (sprite_t *sprite, int x, int y, int w, int h)

  La fonction initialise la position du sprite.
- void init_walls (world_t *world)

  La fonction initialise la position des murs (en fonction du niveau)
- void init_ovnis (world_t *world)

  La fonction initialise la position des ovnis (en fonction du niveau)
- void init_data (world_t *world)

  La fonction initialise les données du monde (en fonction du niveau)
- void blink_splash_text (world_t *world, unsigned int timeSinceLaunch)

  Fait clignoter le texte "PRESS SPACE TO START!" en fonction de la constante SPLASH_TEXT_TIME.
- int sprites_collide (sprite_t *sp1, sprite_t *sp2)

  La fonction renvoie l'état de collision de 2 sprites (0 ou 1)
- void handle_sprites_collision (world_t *world, sprite_t *sp1, sprite_t *sp2, int make_disappear, int make_win)

  La fonction gère les collisions entre deux sprites.
- int is_finished_level (world_t *world)

  Renvoie si le niveau est terminé ou non.
- int is_finished_game (world_t *world)

  Renvoie si le jeu est terminé ou non.
- void update_time (world_t *world, unsigned int actualTime, unsigned int *lastTimeCheck)

  Met à jour le temps écoulé
- void update_finishline (world_t *world)

  Met à jour la position de la ligne d'arrivée + gère les collisions.
- void update_walls (world_t *world)

  Met à jour la position des murs de météorites + gère les collisions (en fonction du niveau)
- void update_ovnis (world_t *world)

  Met à jour la position des ovnis + gère les collisions (en fonction du niveau)
- void shoot_missile (world_t *world)

  Tire un missile à partir du vaisseau.
- void update_missiles (world_t *world)

  Met à jour la position du missile + gère les collisions (en fonction du niveau)
- void update_missileCooldown (world_t *world, unsigned int actualTime, unsigned int *lastMissileCooldown↩ Check)

  Met à jour le temps restant au cooldown du tir de missile du vaisseau.
- void update_data (world_t *world, ressources_t *ressources)

  La fonction met à jour les données en tenant compte de la physique du monde.

4.7.1 Detailed Description

Header de logique.c.

Author

Valentini Andréa & Viale Romain

4.7.2 Function Documentation

4.7.2.1 blink_splash_text()

```
void blink_splash_text (
            world_t * world,
            unsigned int timeSinceLaunch )
```

Fait clignoter le texte "PRESS SPACE TO START!" en fonction de la constante SPLASH_TEXT_TIME.

Parameters

world	les données du monde
timeSinceLaunch	la durée depuis laquelle le jeu est lancé (en ms)

4.7.2.2 handle_sprites_collision()

```
void handle_sprites_collision (
            world_t * world,
            sprite_t * sp1,
            sprite_t * sp2,
            int make_disappear,
            int make_win )
```

La fonction gère les collisions entre deux sprites.

Parameters

world	les données du monde
sp1	le premier sprite
sp2	le deuxième sprite
make_disappear	1 si sp1 doit disparaître en cas de collision, 2 si sp1 et sp2 doivent disparaitre, 0 sinon
make_win	1 si la collision déclenche la victoire, 0 si défaite, autre valeur entière si ça ne doit rien faire
ressources	les ressources du jeu

4.7.2.3 init_data()

```
void init_data (
            world_t * world )
```

La fonction initialise les données du monde (en fonction du niveau)

Parameters

world	les données du monde

4.7.2.4 init_ovnis()

```
void init_ovnis (
            world_t * world )
```

La fonction initialise la position des ovnis (en fonction du niveau)

Parameters

world	les données du monde

4.7.2.5 init_sprite()

```
void init_sprite (
            sprite_t * sprite,
            int x,
            int y,
            int w,
            int h )
```

La fonction initialise la position du sprite.

Parameters

sprite	le sprite du jeu
x	la position x du sprite
y	la position y du sprite
w	la largeur du sprite
h	la hauteur du sprite

4.7.2.6 init_walls()

```
void init_walls (
            world_t * world )
```

La fonction initialise la position des murs (en fonction du niveau)

Parameters

world	les données du monde

4.7.2.7 is_finished_game()

```
int is_finished_game (
            world_t * world )
```

Renvoie si le jeu est terminé ou non.

Parameters

world	les données du monde

4.7.2.8 is_finished_level()

```
int is_finished_level (
            world_t * world )
```

Renvoie si le niveau est terminé ou non.

Parameters

world	les données du monde

4.7.2.9 shoot_missile()

```
void shoot_missile (
            world_t * world )
```

Tire un missile à partir du vaisseau.

Parameters

world	les données du monde

4.7.2.10 sprites_collide()

```
int sprites_collide (
            sprite_t * sp1,
            sprite_t * sp2 )
```

La fonction renvoie l'état de collision de 2 sprites (0 ou 1)

Parameters

sp1	le premier sprite
sp2	le deuxième sprite

4.7.2.11 update_data()

```
void update_data (
            world_t * world,
            ressources_t * ressources )
```

La fonction met à jour les données en tenant compte de la physique du monde.

Parameters

world	les données du monde
ressources	les ressources du jeu

4.7.2.12 update_finishline()

```
void update_finishline (
            world_t * world )
```

Met à jour la position de la ligne d'arrivée + gère les collisions.

Parameters

world	les données du monde

4.7.2.13 update_missileCooldown()

```
void update_missileCooldown (
            world_t * world,
            unsigned int actualTime,
            unsigned int * lastMissileCooldownCheck )
```

Met à jour le temps restant au cooldown du tir de missile du vaisseau.

Parameters

world	les données du monde
actualTime	le temps écoulé depuis le lancement du jeu (en ms)
lastMissileCooldownCheck	le temps écoulé au dernier check (en ms)

4.7.2.14 update_missiles()

```
void update_missiles (
            world_t * world )
```

Met à jour la position du missile + gère les collisions (en fonction du niveau)

Parameters

world	les données du monde

4.7.2.15 update_ovnis()

```
void update_ovnis (
            world_t * world )
```

Met à jour la position des ovnis + gère les collisions (en fonction du niveau)

Parameters

world	les données du monde

4.7.2.16 update_time()

```
void update_time (
            world_t * world,
```

```
            unsigned int actualTime,
            unsigned int * lastTimeCheck )
```

Met à jour le temps écoulé

Parameters

world	les données du monde
actualTime	le temps écoulé depuis le lancement du jeu (en ms)
lastTimeCheck	le temps écoulé au dernier check (en ms)

4.7.2.17 update_walls()

```
void update_walls (
            world_t * world )
```

Met à jour la position des murs de météorites + gère les collisions (en fonction du niveau)

Parameters

world	les données du monde

4.8 logique.h

Go to the documentation of this file.

```
00001
00007 #ifndef LOGIQUE_H
00008 #define LOGIQUE_H
00009
00010 #include "sdl2-light.h"
00011
00015 typedef struct sprite_s{
00016     float x;
00017     float y;
00018     float w;
00019     float h;
00020 } sprite_t;
00021
00025  typedef struct world_s{
00026     sprite_t splash_text;
00027     sprite_t spacecraft;
00028     sprite_t ligne_arrivee;
00029     sprite_t walls_level_1[6];
00030     sprite_t walls_level_2[50];
00031     sprite_t ovnis_level_1[3];
00032     sprite_t ovnis_level_2[30];
00033     sprite_t missiles[10];
00034     sprite_t invisible_limit;
00035     int game_status;
00036     float vy;
00037     int gameover;
00038     int win;
00039     float time;
00040     unsigned int missileCooldown;
00041 } world_t;
00042
00043 #include "affichage.h" // plus bas pour éviter les dépendances circulaires
00044
00053 void init_sprite(sprite_t *sprite, int x, int y, int w, int h);
00054
00059 void init_walls(world_t* world);
00060
00065 void init_ovnis(world_t* world);
```

```
00066
00071 void init_data(world_t * world);
00072
00078 void blink_splash_text(world_t* world, unsigned int timeSinceLaunch);
00079
00085 int sprites_collide(sprite_t* sp1, sprite_t* sp2);
00086
00096 void handle_sprites_collision(world_t* world, sprite_t* sp1, sprite_t* sp2, int make_disappear, int
      make_win);
00097
00102 int is_finished_level(world_t *world);
00103
00108 int is_finished_game(world_t* world);
00109
00116 void update_time(world_t* world, unsigned int actualTime, unsigned int* lastTimeCheck);
00117
00122 void update_finishline(world_t* world);
00123
00128 void update_walls(world_t* world);
00129
00134 void update_ovnis(world_t* world);
00135
00140 void shoot_missile(world_t* world);
00141
00146 void update_missiles(world_t* world);
00147
00154 void update_missileCooldown(world_t* world, unsigned int actualTime, unsigned int*
      lastMissileCooldownCheck);
00155
00161 void update_data(world_t *world, ressources_t* ressources);
00162
00163 #endif
```

4.9 main.c File Reference

Programme principal du jeu.

```
#include "logique.h"
#include "affichage.h"
#include "constantes.h"
```

Include dependency graph for main.c:

Functions

- void init (SDL_Window **window, SDL_Renderer **renderer, ressources_t *ressources, world_t *world)

  Fonction qui initialise le jeu: initialisation de la partie graphique (SDL), chargement des ressources, initialisation des données.
- void skip_to_next_stage (world_t *world)

  Fait passer le jeu au niveau suivant.
- void handle_events (SDL_Event *event, world_t *world)

  Fonction qui gère les évènements ayant eu lieu et qui n'ont pas encore été traités.
- void clean (SDL_Window *window, SDL_Renderer *renderer, ressources_t *ressources, world_t *world)

  Fonction qui nettoie le jeu: nettoyage de la partie graphique (SDL), nettoyage des ressources, nettoyage des données.
- int **main** (int argc, char *args[])

  Le programme principal qui lance et gère le déroulement du jeu.

4.9.1 Detailed Description

Programme principal du jeu.

Author

Valentini Andréa & Viale Romain

4.9.2 Function Documentation

4.9.2.1 clean()

```
void clean (
            SDL_Window * window,
            SDL_Renderer * renderer,
            ressources_t * ressources,
            world_t * world )
```

Fonction qui nettoie le jeu: nettoyage de la partie graphique (SDL), nettoyage des ressources, nettoyage des données.

Parameters

window	la fenêtre du jeu
renderer	le renderer
ressources	les ressources
world	le monde

4.9.2.2 handle_events()

```
void handle_events (
            SDL_Event * event,
            world_t * world )
```

Fonction qui gère les évènements ayant eu lieu et qui n'ont pas encore été traités.

Parameters

event	paramètre qui contient les événements
world	les données du monde

4.9.2.3 init()

```
void init (
            SDL_Window ** window,
            SDL_Renderer ** renderer,
            ressources_t * ressources,
            world_t * world )
```

Fonction qui initialise le jeu: initialisation de la partie graphique (SDL), chargement des ressources, initialisation des données.

Parameters

window	la fenêtre du jeu
renderer	le renderer
ressources	les ressources du jeu
world	le monde du jeu

4.9.2.4 skip_to_next_stage()

```
void skip_to_next_stage (
            world_t * world )
```

Fait passer le jeu au niveau suivant.

Parameters

world	les données du monde

4.10 sdl2-light.c File Reference

sur-couche de SDL2 pour simplifier son utilisation pour le projet

```
#include "sdl2-light.h"
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for sdl2-light.c:

Functions

- int init_sdl (SDL_Window **window, SDL_Renderer **renderer, int width, int height)

  La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.
- SDL_Texture * load_image (const char path[ ], SDL_Renderer *renderer)

  La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.
- void apply_texture (SDL_Texture *texture, SDL_Renderer *renderer, int x, int y)

  La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.
- void clean_texture (SDL_Texture *texture)

  La fonction nettoie une texture en mémoire.
- void clear_renderer (SDL_Renderer *renderer)

  La fonction vide le contenu graphique du renderer lié à l'écran de jeu.
- void update_screen (SDL_Renderer *renderer)

  La fonction met à jour l'écran avec le contenu du renderer.
- void pause (int time)

  La fonction met le programme en pause pendant un laps de temps.
- void clean_sdl (SDL_Renderer *renderer, SDL_Window *window)

  La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

4.10.1 Detailed Description

sur-couche de SDL2 pour simplifier son utilisation pour le projet

Author

Mathieu Constant

Version

0.2

Date

10 mars 2021

4.10.2 Function Documentation

4.10.2.1 apply_texture()

```
void apply_texture (
            SDL_Texture * texture,
            SDL_Renderer * renderer,
            int x,
            int y )
```

La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.

Parameters

texture	la texture que l'on va appliquer
renderer	le renderer qui va recevoir la texture
x	l'abscisse sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)
y	l'ordonnée sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)

4.10.2.2 clean_sdl()

```
void clean_sdl (
            SDL_Renderer * renderer,
            SDL_Window * window )
```

La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

Parameters

renderer	le renderer à nettoyer
window	la fenêtre à nettoyer

4.10.2.3 clean_texture()

```
void clean_texture (
            SDL_Texture * texture )
```

La fonction nettoie une texture en mémoire.

Parameters

texture	la texture à nettoyer

4.10.2.4 clear_renderer()

```
void clear_renderer (
            SDL_Renderer * renderer )
```

La fonction vide le contenu graphique du renderer lié à l'écran de jeu.

Parameters

renderer	le renderer de l'écran

4.10.2.5 init_sdl()

```
int init_sdl (
            SDL_Window ** window,
            SDL_Renderer ** renderer,
            int width,
            int height )
```

La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.

Parameters

window	la fenêtre du jeu
renderer	le renderer
width	largeur de l'écran de jeu
height	hauteur de l'écran de jeu

Returns

-1 en cas d'erreur, 0 sinon

4.10.2.6 load_image()

```
SDL_Texture * load_image (
            const char path[],
            SDL_Renderer * renderer )
```

La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.

Parameters

path	est le chemin du fichier image. Le fichier doit être obligatoirement du BMP.
renderer	le renderer

Returns

la surface SDL contenant l'image avec la couleur RGB (255,0,255) rendue transparente. Elle renvoie NULL si le chargement a échoué (ex. le fichier path n'existe pas)

4.10.2.7 pause()

```
void pause (
            int time )
```

La fonction met le programme en pause pendant un laps de temps.

Parameters

time	ce laps de temps en milliseconde

4.10.2.8 update_screen()

```
void update_screen (
            SDL_Renderer * renderer )
```

La fonction met à jour l'écran avec le contenu du renderer.

Parameters

renderer	le renderer de l'écran

4.11 sdl2-light.h File Reference

en-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet

`#include <SDL2/SDL.h>`
Include dependency graph for sdl2-light.h: This graph shows which files directly or indirectly include this file:

Functions

- void clean_sdl (SDL_Renderer ∗renderer, SDL_Window ∗window)

  La fonction nettoie le renderer et la fenêtre du jeu en mémoire.
- SDL_Texture ∗ load_image (const char path[ ], SDL_Renderer ∗renderer)

La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.

- int init_sdl (SDL_Window **window, SDL_Renderer **renderer, int width, int height)

  La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.
- void clean_texture (SDL_Texture *texture)

  La fonction nettoie une texture en mémoire.
- void apply_texture (SDL_Texture *texture, SDL_Renderer *renderer, int x, int y)

  La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.
- void clear_renderer (SDL_Renderer *renderer)

  La fonction vide le contenu graphique du renderer lié à l'écran de jeu.
- void update_screen (SDL_Renderer *renderer)

  La fonction met à jour l'écran avec le contenu du renderer.
- void pause (int time)

  La fonction met le programme en pause pendant un laps de temps.

4.11.1 Detailed Description

en-tête du module correspondant à une sur-couche de SDL2 pour simplifier son utilisation pour le projet

Author

Mathieu Constant

Version

0.2

Date

10 mars 2021

4.11.2 Function Documentation

4.11.2.1 apply_texture()

```
void apply_texture (
            SDL_Texture * texture,
            SDL_Renderer * renderer,
            int x,
            int y )
```

La fonction permet d'appliquer une texture sur le renderer à une position donnée. La hauteur et la largeur est la même que celle de la texture.

Parameters

texture	la texture que l'on va appliquer
renderer	le renderer qui va recevoir la texture
x	l'abscisse sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)
y	l'ordonnée sur le renderer de l'endroit où est appliquée texture (point en haut à gauche de la surface)

4.11.2.2 clean_sdl()

```
void clean_sdl (
            SDL_Renderer * renderer,
            SDL_Window * window )
```

La fonction nettoie le renderer et la fenêtre du jeu en mémoire.

Parameters

renderer	le renderer à nettoyer
window	la fenêtre à nettoyer

4.11.2.3 clean_texture()

```
void clean_texture (
            SDL_Texture * texture )
```

La fonction nettoie une texture en mémoire.

Parameters

texture	la texture à nettoyer

4.11.2.4 clear_renderer()

```
void clear_renderer (
            SDL_Renderer * renderer )
```

La fonction vide le contenu graphique du renderer lié à l'écran de jeu.

Parameters

renderer	le renderer de l'écran

4.11.2.5 init_sdl()

```
int init_sdl (
            SDL_Window ** window,
            SDL_Renderer ** renderer,
            int width,
            int height )
```

La fonction initialise la SDL. Elle crée la fenêtre du jeu ainsi que le renderer.

Parameters

window	la fenêtre du jeu

Parameters

renderer	le renderer
width	largeur de l'écran de jeu
height	hauteur de l'écran de jeu

Returns

-1 en cas d'erreur, 0 sinon

4.11.2.6 load_image()

```
SDL_Texture * load_image (
            const char path[],
            SDL_Renderer * renderer )
```

La fonction charge une image et renvoie la texture correspondante où la couleur RGB (255, 0, 255) est rendue transparente.

Parameters

path	est le chemin du fichier image. Le fichier doit être obligatoirement du BMP.
renderer	le renderer

Returns

la surface SDL contenant l'image avec la couleur RGB (255,0,255) rendue transparente. Elle renvoie NULL si le chargement a échoué (ex. le fichier path n'existe pas)

4.11.2.7 pause()

```
void pause (
            int time )
```

La fonction met le programme en pause pendant un laps de temps.

Parameters

time	ce laps de temps en milliseconde

4.11.2.8 update_screen()

```
void update_screen (
            SDL_Renderer * renderer )
```

La fonction met à jour l'écran avec le contenu du renderer.

Parameters

renderer	le renderer de l'écran

4.12 sdl2-light.h

Go to the documentation of this file.

```
00001
00010 #ifndef __SDL2_LIGHT__H__
00011 #define __SDL2_LIGHT__H__
00012
00013 #include <SDL2/SDL.h>
00014
00015
00022 void clean_sdl(SDL_Renderer *renderer,SDL_Window *window);
00023
00033 SDL_Texture *load_image(const char path[], SDL_Renderer *renderer);
00034
00035
00046 int init_sdl(SDL_Window **window, SDL_Renderer **renderer, int width, int height);
00047
00053 void clean_texture(SDL_Texture *texture);
00054
00055
00064 void apply_texture(SDL_Texture *texture,SDL_Renderer *renderer,int x, int y);
00065
00066
00067
00073 void clear_renderer(SDL_Renderer *renderer);
00074
00080 void update_screen(SDL_Renderer *renderer);
00081
00088 void pause(int time);
00089
00090 #endif
```

4.13 sdl2-ttf-light.h

```
00001 #ifndef __SDL2_TTF_LIGHT__H
00002 #define __SDL2_TTF_LIGHT__H
00003
00004 #include <SDL2/SDL.h>
00005 #include <SDL2/SDL_ttf.h>
00006
00007
00008 void init_ttf();
00009 TTF_Font * load_font(const char* path, int font_size);
00010 void apply_text(SDL_Renderer *renderer,int x, int y, int w, int h, const char *text, TTF_Font *font);
00011 void clean_font(TTF_Font * font);
00012
00013 #endif
```

Index

affichage.c, 11
affichage.h, 11
    apply_background_level_1, 12
    apply_background_level_2, 13
    apply_level_1_screen, 13
    apply_level_2_screen, 13
    apply_ovnis, 13
    apply_sprites1, 14
    apply_startscreen, 14
    apply_texts, 14
    apply_walls, 15
    clean_ressources, 15
    init_textures, 15
    refresh_graphics, 15
apply_background_level_1
    affichage.h, 12
apply_background_level_2
    affichage.h, 13
apply_level_1_screen
    affichage.h, 13
apply_level_2_screen
    affichage.h, 13
apply_ovnis
    affichage.h, 13
apply_sprites1
    affichage.h, 14
apply_startscreen
    affichage.h, 14
apply_texts
    affichage.h, 14
apply_texture
    sdl2-light.c, 35
    sdl2-light.h, 38
apply_walls
    affichage.h, 15

background
    ressources_s, 5
background2
    ressources_s, 5
blink_splash_text
    logique.c, 19
    logique.h, 25

clean
    main.c, 33
clean_ressources
    affichage.h, 15
clean_sdl
    sdl2-light.c, 35
    sdl2-light.h, 39
clean_texture
    sdl2-light.c, 35
    sdl2-light.h, 39
clear_renderer
    sdl2-light.c, 36
    sdl2-light.h, 39
constantes.h, 16

font
    ressources_s, 6

game_status
    world_s, 9
gameover
    world_s, 9

h
    sprite_s, 7
handle_events
    main.c, 33
handle_sprites_collision
    logique.c, 19
    logique.h, 26

init
    main.c, 33
init_data
    logique.c, 20
    logique.h, 26
init_ovnis
    logique.c, 20
    logique.h, 26
init_sdl
    sdl2-light.c, 36
    sdl2-light.h, 39
init_sprite
    logique.c, 20
    logique.h, 26
init_textures
    affichage.h, 15
init_walls
    logique.c, 21
    logique.h, 27
invisible_limit
    world_s, 9
is_finished_game
    logique.c, 21
    logique.h, 27
is_finished_level

logique.c, 21
logique.h, 27

level_1_screen
ressources_s, 6
level_2_screen
ressources_s, 6
ligne_arrivee
ressources_s, 6
world_s, 9
load_image
sdl2-light.c, 36
sdl2-light.h, 40
logique.c, 18
blink_splash_text, 19
handle_sprites_collision, 19
init_data, 20
init_ovnis, 20
init_sprite, 20
init_walls, 21
is_finished_game, 21
is_finished_level, 21
shoot_missile, 21
sprites_collide, 22
update_data, 22
update_finishline, 22
update_missileCooldown, 23
update_missiles, 23
update_ovnis, 23
update_time, 23
update_walls, 24
logique.h, 24
blink_splash_text, 25
handle_sprites_collision, 26
init_data, 26
init_ovnis, 26
init_sprite, 26
init_walls, 27
is_finished_game, 27
is_finished_level, 27
shoot_missile, 28
sprites_collide, 28
update_data, 28
update_finishline, 28
update_missileCooldown, 30
update_missiles, 30
update_ovnis, 30
update_time, 30
update_walls, 31

main.c, 32
clean, 33
handle_events, 33
init, 33
skip_to_next_stage, 34
meteorite
ressources_s, 6
missile
ressources_s, 6
missileCooldown
world_s, 9
missiles
world_s, 9

ovni
ressources_s, 6
ovnis_level_1
world_s, 9
ovnis_level_2
world_s, 10

pause
sdl2-light.c, 37
sdl2-light.h, 40

refresh_graphics
affichage.h, 15
ressources_s, 5
background, 5
background2, 5
font, 6
level_1_screen, 6
level_2_screen, 6
ligne_arrivee, 6
meteorite, 6
missile, 6
ovni, 6
spacecraft, 6
splash_text, 7
start_screen, 7

sdl2-light.c, 34
apply_texture, 35
clean_sdl, 35
clean_texture, 35
clear_renderer, 36
init_sdl, 36
load_image, 36
pause, 37
update_screen, 37
sdl2-light.h, 37
apply_texture, 38
clean_sdl, 39
clean_texture, 39
clear_renderer, 39
init_sdl, 39
load_image, 40
pause, 40
update_screen, 40
shoot_missile
logique.c, 21
logique.h, 28
skip_to_next_stage
main.c, 34
spacecraft
ressources_s, 6
world_s, 10
splash_text

ressources_s, 7
world_s, 10
sprite_s, 7
h, 7
w, 7
x, 8
y, 8
sprites_collide
logique.c, 22
logique.h, 28
start_screen
ressources_s, 7

time
world_s, 10

update_data
logique.c, 22
logique.h, 28
update_finishline
logique.c, 22
logique.h, 28
update_missileCooldown
logique.c, 23
logique.h, 30
update_missiles
logique.c, 23
logique.h, 30
update_ovnis
logique.c, 23
logique.h, 30
update_screen
sdl2-light.c, 37
sdl2-light.h, 40
update_time
logique.c, 23
logique.h, 30
update_walls
logique.c, 24
logique.h, 31

vy
world_s, 10

w
sprite_s, 7
walls_level_1
world_s, 10
walls_level_2
world_s, 10
win
world_s, 10
world_s, 8
game_status, 9
gameover, 9
invisible_limit, 9
ligne_arrivee, 9
missileCooldown, 9
missiles, 9
ovnis_level_1, 9
ovnis_level_2, 10
spacecraft, 10
splash_text, 10
time, 10
vy, 10
walls_level_1, 10
walls_level_2, 10
win, 10

x
sprite_s, 8

y
sprite_s, 8